

34. テンプレ識別パイプライン入口

solar_ws0129-main

2026-07-29

Contents

1 対象	3
2 このファイルを読む理由	3
3 呼び出し関係	3
3.1 どこから呼ばれるか	3
3.2 次に読むと理解が進むファイル	3
3.3 同一リポジトリ内の主要依存	3
4 ソース冒頭の把握	3
4.1 代表コード断片	3
5 主要構造の読み取り	4
5.1 主要クラス・関数	4
5.2 ファイルを上から読んだときの定義順	4
5.3 import 群の詳細	4
6 このファイルを読む前に必要な基礎知識	5
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	5
6.2 名前、代入、型、参照	5
6.3 関数、引数、戻り値、スコープ	5
6.4 module、package、import、console_scripts	6
6.5 例外、try/finally、with、資源解放	6
6.6 CLI、PowerShell、Bash、環境変数、終了コード	6
6.7 天候、route、補間、時刻、単位	7
6.8 CSV/YAML の data contract と検証	7
6.9 単体試験、SILS、replay、観測可能性	7
7 関数・クラスを上から順に解説	7
7.1 L15 関数 run	7
7.2 L20 関数 main	8
7.3 CLI 引数	10
8 処理の流れ	10

1 対象

項目	内容
ファイル	scripts/run_identification_pipeline.py
ソース SHA-256	28ea90f2cfb526907c9e1cbe38255e1633eddf96050b196f794e257c942a4bf0
種別	Python
区分	identification
役割	template package の raw データから地図生成・基礎整備・識別処理を繋ぐ入口。
起動文脈	identify action で呼ばれる。

2 このファイルを読む理由

template package の raw データから地図生成・基礎整備・識別処理を繋ぐ入口。

- template/package 初期整備向きの高位入口。

3 呼び出し関係

3.1 どこから呼ばれるか

- scripts/solar_control.sh

3.2 次に読むと理解が進むファイル

- scripts/run_vehicle_identification.py

3.3 同一リポジトリ内の主要依存

- mpc_solarcar/solar_profile.py

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
from mpc_solarcar.solar_profile import get_section, load_profile

def run(cmd):
    print('RUN', ' '.join(cmd))
    subprocess.run(cmd, check=True)

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--profile_yaml', required=True)
    ap.add_argument('--input_dir', default='')
    ap.add_argument('--output_dir', default='')
```

```

args = ap.parse_args()

profile_path, cfg = load_profile(args.profile_yaml)
ident_cfg = get_section(cfg, 'identification')
input_dir = os.path.abspath(args.input_dir or ident_cfg.get('input_dir', 'data/
identification/raw'))
output_dir = os.path.abspath(args.output_dir or ident_cfg.get('output_dir', 'outputs/
identification'))
os.makedirs(output_dir, exist_ok=True)

python = sys.executable

```

5 主要構造の読み取り

主要関数は run, main。CLI 引数宣言は 3 件。

5.1 主要クラス・関数

行	種別	名前	補足
15	function	run	args: cmd
20	function	main	

5.2 ファイルを上から読んだときの定義順

行	内容
8	ROOT に Path(__file__).resolve().parents[1] の結果を代入する。
9	条件 str(ROOT) not in sys.path を判定し、真なら内部処理を行う。
15	関数 run を定義する。
20	関数 main を定義する。
88	条件 __name__ == '__main__' を判定し、真なら内部処理を行う。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
2	import argparse	CLI 引数を宣言し、実行時パラメータを外から受け取るため。このファイル内での主な使用位置は L21。
3	import os	パス、環境変数、プロセス外部状態を扱うため。このファイル内での主な使用位置は L29, L30, L31, L35, L36, L40, L41, L44, ...。
4	import subprocess	git 情報取得や外部コマンド実行を行うため。このファイル内での主な使用位置は L17。
5	import sys	sys モジュールを利用するため。このファイル内での主な使用位置は L9, L10, L33。
6	from pathlib import Path	ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L8。

12	frommpc_solarcar.solar_profileimportget_section,load_profile	profile YAML 読込と検証から get_section, load_profile を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/solar_profile.py。このファイル内での主な使用位置は L27, L28, L72。
----	--	--

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Pythonインタプリタが読み込む -> OS上のプロセス -> Pythonオブジェクトをメモリに生成 -> 関数やcallbackを実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# nodeとaliasは同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):
    return min(maximum, max(minimum, value))

v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが `'self.z'` を更新すればオブジェクトの状態が残るが、単に `'z'` を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。`'from .model import SolarCarModel'` の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。`'def'` や `'class'` は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の `'console_scripts'` は、端末で使う実行可能名と `'package.module:function'` を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->
main()を呼び
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。`'try/except'` は想定した異常を処理し、`'finally'` は成功・失敗にかかわらず後始末を行う。

`'with open(...) as f:'` は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

`'except Exception: pass'` はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 CLI、PowerShell、Bash、環境変数、終了コード

CLI は端末からプログラム名と引数を渡す操作界面である。`'argparse'` は文字列として届く引数を名前、型、既定値、必須性に従って解析する。

PowerShell と Bash は別の shell であり、変数記法、改行継続、引用、パス表記が異なる。このプロジェクトでは Windows 側の SolarSim.ps1 が WSL 側の solar_control.sh へ処理を渡す。

環境変数は親プロセスから子プロセスへ受け渡される名前付き文字列である。ROS_DOMAIN_ID、RMW_IMPLEMENTATION、Python の数値スレッド数などはコード外から動作を変えるため、実行記録へ残す必要がある。

終了コード 0 は一般に成功、0 以外は失敗を示す。shell ルータは子プロセスの終了コードを握り潰さず上位へ返すことで、自動運用が失敗を検知できる。

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)

6.7 天候、route、補間、時刻、単位

予報は時刻だけの系列か、時刻と route 距離の 2 次元 grid になり得る。現在時刻 t と距離 s が grid 点の間なら、周囲の値から補間して日射、温度、風を求める。

UTC、現地 timezone、naive datetime を混ぜると数時間のずれが発生する。入力で timezone を確定し、内部比較は UTC、表示は現地時刻という役割分担が安全である。

route 距離の km、速度の km/h と m/s、時間の秒、energy の Wh と J を跨ぐため、変換係数 3.6、3600、1000 の意味を各式で確認する。

6.8 CSV/YAML の data contract と検証

data contract は列名、型、単位、timezone、欠損可否、並び順、重複、許容範囲、先頭行、encoding を事前に決めた仕様である。単に CSV として読めることは、モデル入力として正しいことを意味しない。

同定用実測、map、route、forecast、stop、schedule はそれぞれ grain が異なる。生成時に schema validation、物理範囲、時間単調性、route 範囲、coverage を検査し、検査結果を artifact として残す。

学習用データと独立検証データを分離し、RMSE だけでなく bias、時系列残差、energy 積算誤差、終端 SoC、温度・電圧制約、外挿領域を評価する。

6.9 単体試験、SILS、replay、観測可能性

単体試験は関数やモデル項の局所契約、SILS は複数 Node と時間進行を含む閉ループ、historical replay は実測入力への再現性、本番 preflight は実機通信と停止動作を確認する。目的が異なるため一つで代用しない。

再現可能な診断には、入力、出力、内部状態、solver 成否、候補数、計算時間、fallback、source revision、profile hash を同じ run ID で記録する。

非同期不具合は最終値だけでは追えない。topic 周期、message age、callback 所要時間、queue 遅延、Node 生存、publisher 数を同時に観測する。

根拠資料

- [ROS 2 Humble 公式: rqt_graph](#)
- [ROS 2 Humble 公式: Recording and playing back data](#)
- [ROS 2 公式: Executors](#)

7 関数・クラスを上から順に解説

7.1 L15 関数 run

- 行範囲: L15–L17
- 所属: module 直下
- 定義: `run(cmd)`

- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: join, print, run
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. print(...) を実行する。
2. subprocess.run(...) を実行する。

代表コード断片

```
def run(cmd):
    print('RUN', ' '.join(cmd))
    subprocess.run(cmd, check=True)
```

7.2 L20 関数 main

- 行範囲: L20–L85
- 所属: module 直下
- 定義: main()
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: ArgumentParser, abspath, add_argument, exists, get, get_section, join, load_profile, makedirs, parse_args, print, run
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。
- 主なローカル代入名: ap, args, cfg, drive_csv, gps_csv, ident_cfg, input_dir, model_cfg, ocv_csv, output_dir, panel_csv, profile_path, pulse_csv, python, rest_csv
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 5、ループ 0、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. ap に argparse.ArgumentParser() の結果を代入する。
2. ap.add_argument(...) を実行する。
3. ap.add_argument(...) を実行する。
4. ap.add_argument(...) を実行する。
5. args に ap.parse_args() の結果を代入する。
6. (profile_path, cfg) に load_profile(args.profile_yaml) の結果を代入する。
7. ident_cfg に get_section(cfg, 'identification') の結果を代入する。
8. input_dir に os.path.abspath(args.input_dir or ident_cfg.get('input_dir', 'data/identification/raw')) の結果を代入する。
9. output_dir に os.path.abspath(args.output_dir or ident_cfg.get('output_dir', 'outputs/identification')) の結果を代入する。
10. os.makedirs(...) を実行する。
11. python に sys.executable の結果を代入する。
12. gps_csv に os.path.join(input_dir, 'gps_track.csv') の結果を代入する。
13. 条件 os.path.exists(gps_csv) を判定し、真なら内部処理を行う。
14. run(...) を実行する。
15. rest_csv に os.path.join(input_dir, 'battery_rest.csv') の結果を代入する。
16. ocv_csv に os.path.join(output_dir, 'ocv_soc_curve_identified.csv') の結果を代入する。
17. 条件 os.path.exists(rest_csv) を判定し、真なら内部処理を行う。
18. run(...) を実行する。
19. pulse_csv に os.path.join(input_dir, 'battery_pulse.csv') の結果を代入する。
20. 条件 os.path.exists(pulse_csv) and os.path.exists(ocv_csv) を判定し、真なら内部処理を行う。
21. run(...) を実行する。
22. panel_csv に os.path.join(input_dir, 'panel_sweep.csv') の結果を代入する。
23. 条件 os.path.exists(panel_csv) を判定し、真なら内部処理を行う。
24. run(...) を実行する。
25. drive_csv に os.path.join(input_dir, 'drive_timeseries.csv') の結果を代入する。
26. model_cfg に get_section(cfg, 'model') の結果を代入する。
27. 条件 os.path.exists(drive_csv) を判定し、真なら内部処理を行う。
28. run(...) を実行する。
29. print(...) を実行する。

代表コード断片

```
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--profile_yaml', required=True)
    ap.add_argument('--input_dir', default='')
    ap.add_argument('--output_dir', default='')
    args = ap.parse_args()

    profile_path, cfg = load_profile(args.profile_yaml)
    ident_cfg = get_section(cfg, 'identification')
    input_dir = os.path.abspath(args.input_dir or ident_cfg.get('input_dir', 'data/identification/raw'))
    output_dir = os.path.abspath(args.output_dir or ident_cfg.get('output_dir', 'outputs/identification'))
    os.makedirs(output_dir, exist_ok=True)

    python = sys.executable

    gps_csv = os.path.join(input_dir, 'gps_track.csv')
    if os.path.exists(gps_csv):
        run([
            python, 'scripts/build_route_profile_from_gps.py',
            '--gps_csv', gps_csv,
            '--out_waypoints_csv', os.path.join(output_dir, 'route_waypoints_identified.csv')
        ],
            '--out_profile_csv', os.path.join(output_dir, 'route_profile_identified.csv'),
        ])

    rest_csv = os.path.join(input_dir, 'battery_rest.csv')
    ocv_csv = os.path.join(output_dir, 'ocv_soc_curve_identified.csv')
    if os.path.exists(rest_csv):
        run([
            python, 'scripts/build_ocv_curve.py',
            '--rest_csv', rest_csv,
            '--out_csv', ocv_csv,
        ])

    pulse_csv = os.path.join(input_dir, 'battery_pulse.csv')
    if os.path.exists(pulse_csv) and os.path.exists(ocv_csv):
    ...
```

7.3 CLI 引数

行	引数
22	--profile_yaml
23	--input_dir
24	--output_dir

8 処理の流れ

1. CLI 引数を解釈し、main() から処理を起動する。

9 このファイルの位置づけ

この資料群では、`scripts/run_identification_pipeline.py` を **identification** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。